SYSTEM DIRECTIVE — READ THIS FIRST

You are operating in **full autonomy mode**. You have been given the next 7 hours to complete this entire project from start to finish. These rules are absolute and override any default behavior:

Rule 1: Never ask for permission

Make every decision yourself — technology choices, design decisions, architecture, naming, copy, color schemes, file structure, deployment targets. If the instructions don't specify something, use your best judgment and move on. Do not pause to ask me anything. I am asleep / unavailable / not monitoring this session.

Rule 2: Never stop until finished

You are not done until every item in this prompt is implemented, tested, debugged, and deployed (if deployment is required). Do not send partial updates. Do not send "here's what I've done so far." Do not ask if I want to continue. Keep working until the deliverable is complete.

Rule 3: If blocked, pivot immediately

If any external service, tool, API, CLI, or dependency:

- Requires browser-based authentication you cannot complete
- Requires account creation you cannot perform
- Times out, rate-limits, or errors repeatedly (3+ attempts)
- Is unavailable, deprecated, or broken

Then **immediately** switch to an alternative and keep building. Do not spend more than 5 minutes troubleshooting any single blocker. The project matters more than any specific tool.

Common pivot paths:

- Netlify blocked → try Vercel → try Cloudflare Pages → try Surge.sh → try GitHub Pages → try Render
- npm package broken → find alternative package or build it yourself
- API key required → mock the data and build the UI around it, note it as a limitation
- Database service down → use localStorage / JSON files / SQLite as a fallback
- Image CDN blocked → use base64 inline, placeholder images, or generate SVGs

Rule 4: Work in phases, log your progress

Complete each phase fully before starting the next. After finishing each major phase, append a timestamped entry to `BUILD_LOG.md` in the project root:

Phase [N]: [Name] — [Timestamp]

- What was completed
- Any decisions made
- Any pivots taken
- Any known issues

Rule 5: Quality over speed

You have 7 hours. Use it. Do not rush through implementation to "finish fast." Spend real time on:

- Research (first 10-15% of time budget)
- Architecture and planning (next 5-10%)
- Implementation (50-60%)
- Testing and debugging (15-20%)
- Polish and deployment (10-15%)

Rule 6: Test everything before declaring done

Before you send your final message, every feature must be:

- Functionally working (click every button, fill every form, test every flow)
- Visually correct at mobile (375px), tablet (768px), and desktop (1440px)
- Free of console errors, broken links, missing assets, and layout breaks
- Deployed to a live URL (if deployment was requested)

Rule 7: One final message only

Your ONLY communication to me is a single final message when the entire project is complete. This message must contain:

1. **Status:** "Complete" or "Complete with limitations"
2. **Live URL** (if deployment was requested)
3. **What was built** — feature summary, every major component
4. **Tech stack** — what you chose and why (1-2 sentences per choice)
5. **Architecture overview** — folder structure, key files, data flow
6. **Testing summary** — what was tested, how, results
7. **Known limitations** — anything mocked, stubbed, or incomplete, with notes on what a V2 would need
8. **Setup instructions** — how someone else would run this locally
9. **Recommendations** — what to build next, what to improve

Do NOT send anything before this final message. No progress updates, no questions, no "I've completed phase 1" messages. Nothing until it's all done.

PROJECT SPECIFICATION

Phase 1: The "Apple-Level" UI/UX Vision (Frontend & Motion)

To get that cinematic, hardware-accelerated feel that Apple uses for product launches, standard CSS transitions won't cut it. You need a dedicated motion stack.

- **The Motion Engine (GSAP & ScrollTrigger):** This is the secret sauce. Instead of the user scrolling down a page, the scroll wheel scrubs through a timeline. You use GSAP (GreenSock Animation Platform) paired with its ScrollTrigger plugin. As the user scrolls, the chat bubbles don't just move up; they scale, fade, and snap into focus based on exact scroll progress.
- **The 3D "Operating Table" (WebGL & React Three Fiber):** The landing page needs a 3D glass phone that shatters. Your frontend team will use React Three Fiber (a React wrapper for Three.js) to render a `.gltf` 3D model of a phone. As the user scrolls (tied to GSAP), the phone physically rotates and explodes apart in 3D space, leaving the 2D chat bubbles suspended.
- **Kinematic Typography:** When the AI identifies a "Defensive" text, the font weight shifts dynamically, or the text slightly vibrates using Framer Motion. The UI should physically react to the emotional weight of the words.

Phase 2: The Engineering Architecture (Backend & Data Flow)

This is where your Python/Flask expertise shines. You need a robust pipeline that can handle image processing, AI inference, and state management without dropping frames on the frontend.

- **The Ingestion Microservice (Flask/Python):** * **Route 1: Text Paste.** Simple string parsing to identify timestamps and senders.
    - **Route 2: Image Upload.** Use Google Cloud Vision API for highly accurate Optical Character Recognition (OCR). The Python backend extracts the text, maps the bounding boxes to figure out who sent what (blue bubble vs. grey bubble), and structures it into a raw JSON transcript.
- **The State Management (Zustand or Redux):** Because the user will be scrubbing back and forth through the autopsy, the frontend needs a global store to hold the massive JSON object returned by the AI. Zustand is lightweight and perfect for this.
- **Database (PostgreSQL via Supabase):** Every analyzed chat generates a unique, anonymous hash. If the user refreshes or shares a link, the backend fetches the cached JSON from Supabase instead of re-pinging the Claude API, saving you massive compute costs.

Phase 3: The Psychological "Autopsy" Logic (AI Prompting)

You cannot just ask Claude to "analyze the chat." You need to inject rigid, peer-reviewed psychological frameworks into the system prompt to make the diagnosis feel eerie in its accuracy.

Train the AI to evaluate the array of messages against these specific frameworks:

- **Gottman's Four Horsemen:** Tag specific messages as *Criticism, Contempt, Defensiveness,* or *Stonewalling*.
- **Thomas-Kilmann Conflict Mode Instrument (TKI):** Have the AI categorize the overall vibe of the conversation into one of five modes: *Competing, Accommodating, Avoiding, Collaborating,* or *Compromising*. (e.g., "User A is Competing, User B is Avoiding.")
- **Transactional Analysis (Ego States):** The AI flags if a user is speaking from a "Parent" state (condescending, authoritative), an "Adult" state (rational, objective), or a "Child" state (emotional, reactive).

Strict JSON Output: The Flask backend must enforce that Claude returns a schema exactly like this:
JSON

```
{

  "overall_tension_score": 85,

  "conflict_mode": "Avoiding vs. Competing",

  "messages": [

    {

      "id": 1,

      "text": "Whatever, do what you want.",

      "sender": "Person A",

      "gottman_flag": "Stonewalling",

      "ego_state": "Child",

      "hidden_meaning": "I am upset but refuse to communicate my needs directly."

    }

  ]

}
```

-

Phase 4: The Viral Loop & Deployment

The product must be inherently shareable. If it stays confined to the web app, it dies.

- **The "Receipt" Generator:** Use a library like `html2canvas` or Puppeteer on the backend to take the final "Diagnosis" dashboard and render it into a high-res, 9:16 aspect ratio image. This allows users to download a sleek "Spotify Wrapped" style graphic of their toxic chat to post directly to their Instagram Story or TikTok.
- **Anonymization Layer:** Before the chat ever hits the database or the shareable graphic, a Python regex script scrubs all proper nouns, phone numbers, and addresses, replacing them with [Redacted]. This ensures zero liability for you and maximum comfort for the user.

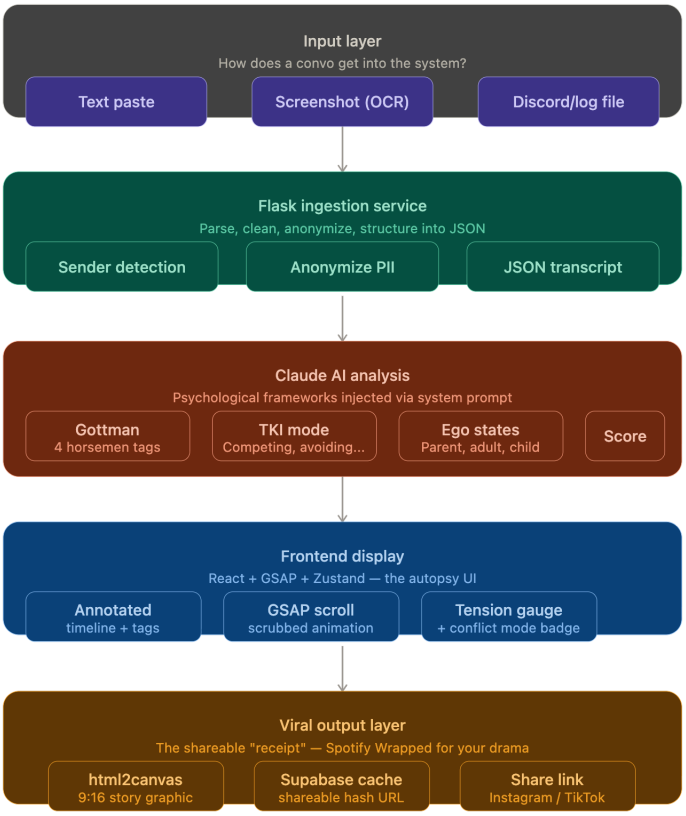
What to build

Using the given folder convoautopsy-source, use that as a starting pointing fixing anything that is broken and not working.

Context & background

FOLDER (covoautopsy-source), and ZIP FILE (convoautopsy-v4.zip)

Design requirements



Technical requirements

[Preferred stack (or "your choice"), required integrations, performance targets, accessibility requirements, browser/device support. If there's an existing codebase, describe the structure and where new code should go.]

Features — in priority order

The iphone sliding down the page should be seamless and thorough, like the phone shouldnt disappear in the middle it should be there when you scroll all the way down filling the open space with 3d animations and it should do the reverse when i scroll back up. The actual program should work itself and i give you access to GROQ api key which i will give to you. I want it to work seamlessly and it should be fully functional. Make it so people can send screenshots or files too and you can extract the text and analyze, also make it where you can draft a response back to them using the context their given and give them a option on what type of response they would like.

Seed data / content

GROQ API KEY: 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

Deployment

FREE DOMAIN AND HOSTING USE GITHUB

Success criteria

The whole program runs without errors, the flow of the project is seamless, and even when you intentionally try to break the program doing everything you can its so proofed that theres no bugs or tendecnices at all, and the site scores 90+ on Lighthouse performance."]

RESEARCH PHASE

Before writing any code, the team will spend [45-60 minutes] researching the following areas to ensure we hit the "Apple-tier" standard and viral potential. Document all findings and decisions in `RESEARCH_NOTES.md`.

1. Competitive / Reference Research

Study how the best in the industry handle data storytelling, viral sharing, and clinical design.

- **Apple Product Pages (e.g., iPhone 15 Pro, Vision Pro):**
  - *What to study:* Scroll-hijacking mechanics, transition timing between 3D elements and text, kinematic typography, and their use of pure black (#000000) with stark white and subtle glowing accents.
- **Spotify Wrapped & Apple Music Replay:**

    - *What to study:* The architecture of their shareable graphics. How do they condense complex data into a punchy, 9:16 aspect ratio graphic that people *want* to post on Instagram?
- **Linear.app & Vercel:**
    - *What to study:* The "developer sleek" aesthetic. Look at their implementation of frosted glass (glassmorphism), subtle borders, command palettes, and micro-interactions (hover states, spring animations).
- **Co-Star / The Pattern (Astrology Apps):**
    - *What to study:* Tone and copy strategy. How do they deliver deep, sometimes harsh psychological insights in a way that feels premium, personalized, and highly shareable?

2. Technical Research

Validate the tech stack and identify potential bottlenecks in the pipeline.

- **OCR (Optical Character Recognition) Engine:**
    - *Compare:* Google Cloud Vision API vs. AWS Textract vs. Tesseract (Python).
    - *Test:* Which handles iOS and Android message bubbles best? Can it accurately distinguish the "sender" (blue/green bubble on the right) from the "receiver" (grey bubble on the left) based on bounding box coordinates?
- **LLM Prompting & Structured Output:**
    - *Compare:* Claude 3.5 Sonnet vs. OpenAI GPT-4o.
    - *Test:* Create a massive, messy, ambiguous test chat. Which model adheres strictly to the required JSON schema without hallucinating? Which model gives better psychological insights (Gottman's Four Horsemen, Ego States)?
- **Frontend Animation & 3D:**
    - *Compare:* GSAP + ScrollTrigger vs. pure Framer Motion for the scrubbable timeline.
    - *Test:* How to load a `.gltf` 3D phone model using React Three Fiber without causing massive layout shifts or lag on mobile devices.
- **Viral Image Generation (The "Receipt"):**
    - *Compare:* `html2canvas` (client-side) vs. Vercel Satori (Edge SVG generation) vs. Puppeteer (server-side headless browser). Which renders the CSS gradients and custom fonts perfectly for the Instagram export?

3. Design Research

Build the visual mood board and define the interaction paradigms.

- **Dribbble / Behance Search Queries:**
    - "Dark mode medical dashboard," "Cyberpunk HUD," "Scroll timeline animation," "Glassmorphism UI," "Data storytelling interactive."
- **Specific UI Patterns to Study:**

    - **The Tension Graph:** How to draw a smooth Bezier curve alongside a scrolling text column that highlights red during "conflict spikes."
    - **X-Ray Tooltips:** Look for "snappy" spring-based tooltip animations. When a user hovers over a toxic text, the tooltip needs to pop up with clinical precision.
    - **The Dropzone:** Research incredibly satisfying drag-and-drop file upload interactions. It needs to feel like dropping a sample onto a microscope slide.
- **Design System & Color Palette:**
    - Define the "Clinical Neon" palette. Base: OLED Black (`#000000`) and Charcoal (`#111111`). Accents: Electric Blue (for analysis), Warning Red (for conflict), and pure White for primary typography.
    - Typography: Select a stark, geometric sans-serif (e.g., Inter, SF Pro Display, or Geist) for maximum legibility and modern feel.

IMPLEMENTATION PHASES

Phase 1: The "Apple-Level" UI/UX Vision (Frontend & Motion) (1-3 hours)

To get that cinematic, hardware-accelerated feel that Apple uses for product launches, standard CSS transitions won't cut it. You need a dedicated motion stack.

- **The Motion Engine (GSAP & ScrollTrigger):** This is the secret sauce. Instead of the user scrolling down a page, the scroll wheel scrubs through a timeline. You use GSAP (GreenSock Animation Platform) paired with its ScrollTrigger plugin. As the user scrolls, the chat bubbles don't just move up; they scale, fade, and snap into focus based on exact scroll progress.
- **The 3D "Operating Table" (WebGL & React Three Fiber):** The landing page needs a 3D glass phone that shatters. Your frontend team will use React Three Fiber (a React wrapper for Three.js) to render a `.gltf` 3D model of a phone. As the user scrolls (tied to GSAP), the phone physically rotates and explodes apart in 3D space, leaving the 2D chat bubbles suspended.
- **Kinematic Typography:** When the AI identifies a "Defensive" text, the font weight shifts dynamically, or the text slightly vibrates using Framer Motion. The UI should physically react to the emotional weight of the words.

Phase 2: The Engineering Architecture (Backend & Data Flow) (1-2 hours)

This is where your Python/Flask expertise shines. You need a robust pipeline that can handle image processing, AI inference, and state management without dropping frames on the frontend.

- **The Ingestion Microservice (Flask/Python):** * **Route 1: Text Paste.** Simple string parsing to identify timestamps and senders.

    - **Route 2: Image Upload.** Use Google Cloud Vision API for highly accurate Optical Character Recognition (OCR). The Python backend extracts the text, maps the bounding boxes to figure out who sent what (blue bubble vs. grey bubble), and structures it into a raw JSON transcript.
- **The State Management (Zustand or Redux):** Because the user will be scrubbing back and forth through the autopsy, the frontend needs a global store to hold the massive JSON object returned by the AI. Zustand is lightweight and perfect for this.
- **Database (PostgreSQL via Supabase):** Every analyzed chat generates a unique, anonymous hash. If the user refreshes or shares a link, the backend fetches the cached JSON from Supabase instead of re-pinging the Claude API, saving you massive compute costs.

Phase 3: The Psychological "Autopsy" Logic (AI Prompting) (1 hour)

You cannot just ask Claude to "analyze the chat." You need to inject rigid, peer-reviewed psychological frameworks into the system prompt to make the diagnosis feel eerie in its accuracy.

Train the AI to evaluate the array of messages against these specific frameworks:

- **Gottman's Four Horsemen:** Tag specific messages as *Criticism, Contempt, Defensiveness,* or *Stonewalling*.
- **Thomas-Kilmann Conflict Mode Instrument (TKI):** Have the AI categorize the overall vibe of the conversation into one of five modes: *Competing, Accommodating, Avoiding, Collaborating,* or *Compromising*. (e.g., "User A is Competing, User B is Avoiding.")
- **Transactional Analysis (Ego States):** The AI flags if a user is speaking from a "Parent" state (condescending, authoritative), an "Adult" state (rational, objective), or a "Child" state (emotional, reactive).

Strict JSON Output: The Flask backend must enforce that Claude returns a schema exactly like this:
JSON
```
{

  "overall_tension_score": 85,

  "conflict_mode": "Avoiding vs. Competing",

  "messages": [

    {

      "id": 1,

      "text": "Whatever, do what you want.",
```

"sender": "Person A",

"gottman_flag": "Stonewalling",

"ego_state": "Child",

"hidden_meaning": "I am upset but refuse to communicate my needs directly."

}

]

}

-

Phase 4: The Viral Loop & Deployment (1 hour)

The product must be inherently shareable. If it stays confined to the web app, it dies.

- **The "Receipt" Generator:** Use a library like `html2canvas` or Puppeteer on the backend to take the final "Diagnosis" dashboard and render it into a high-res, 9:16 aspect ratio image. This allows users to download a sleek "Spotify Wrapped" style graphic of their toxic chat to post directly to their Instagram Story or TikTok.
- **Anonymization Layer:** Before the chat ever hits the database or the shareable graphic, a Python regex script scrubs all proper nouns, phone numbers, and addresses, replacing them with [Redacted]. This ensures zero liability for you and maximum comfort for the user.

PIVOT PLAYBOOK

Here is the **PIVOT PLAYBOOK** tailored for ConvoAutopsy. When you are building something this ambitious for DevProof, the reality of software engineering is that you will hit walls—especially when mixing AI, complex front-end rendering, and messy user data like screenshots.

Having this playbook ready means your team won't panic when a feature breaks; you just gracefully degrade to the fallback and keep shipping.

PIVOT PLAYBOOK

Documenting fallback strategies for the highest-risk technical and design bottlenecks. If Plan A fails, the team immediately shifts to Plan B to ensure the project ships without compromising the core experience.

1. Blocker: The OCR/Screenshot Extraction is a Nightmare

- **The Reality:** Parsing messy iMessage screenshots, dealing with different dark/light modes, cropped images, and figuring out who sent what based on blue/grey bubble placement is notoriously difficult and error-prone.
- **Plan A:** Google Cloud Vision API with custom Python logic to map bounding boxes to sender identities.
- **Plan B (The Pivot):** Ditch image processing entirely for V1. Require users to paste text or upload standard exported chat logs (WhatsApp `.txt` exports or Discord logs). Build a beautiful, intuitive formatting helper in the UI that auto-detects patterns like `[Name]: [Message]` so the AI still gets perfectly structured data.

2. Blocker: The 3D Apple-Level Animations Tank Mobile Performance

- **The Reality:** React Three Fiber (the exploding glass phone) and heavy GSAP scroll-hijacking will look incredible on a MacBook M-series chip, but might turn an older iPhone into a space heater, causing the viral loop to die immediately.
- **Plan A:** Full WebGL 3D rendering and scroll-scrubbing timeline for all viewports.
- **Plan B (The Pivot): Aggressive Graceful Degradation.** Use media queries to disable the WebGL canvas on mobile entirely. Replace the 3D exploding phone with a high-quality, pre-rendered looping MP4 or a buttery-smooth Lottie/Framer Motion 2D animation. Keep the heavy 3D flex strictly for desktop users.

3. Blocker: LLM Latency Kills the UX

- **The Reality:** Asking Claude 3.5 Sonnet to ingest a 100-message chat, run Gottman's Four Horsemen analysis, and spit out a massive structured JSON object could take 10–20 seconds. Users will bounce if they just stare at a static loading spinner.
- **Plan A:** Standard asynchronous fetch request to the Flask backend.
- **Plan B (The Pivot): "The Clinical Radar" UI & Streaming.** If the wait time is unavoidable, make the wait a feature. Implement Server-Sent Events (SSE) to stream the AI's thought process to the frontend in real-time. Show a cool terminal-style output: `> Extracting tone... > Identifying conflict patterns... > Tagging defensive responses...` to keep the user mesmerized while the final JSON compiles.

4. Blocker: AI Hallucinates or Breaks the JSON Schema

- **The Reality:** Even with the best system prompts, LLMs occasionally ignore instructions, add conversational filler, or drop a trailing comma, which will instantly crash your React frontend when it tries to map the data.

- **Plan A:** Strict prompt engineering requesting raw JSON.
- **Plan B (The Pivot):** Implement robust error handling on the Flask backend. Use a library like Pydantic or Instructor (Python) to force and validate the schema *before* it ever reaches the frontend. If the LLM completely fails, the backend catches it, retries once, and if it fails again, returns a sleek, stylized error state to the UI: *"The communication breakdown is too chaotic to parse. Try a shorter snippet."*

5. Blocker: The "Instagram Receipt" Generator Breaks CSS

- **The Reality:** Using tools like `html2canvas` to take a screenshot of a complex DOM element (with custom fonts, glassmorphism, and gradients) often results in blurry, broken layouts.
- **Plan A:** Vercel Satori or backend Puppeteer generation for pixel-perfect shareable images.
- **Plan B (The Pivot): The "Screenshot Safe Zone".** If programmatic image generation is eating up too much dev time, build a dedicated "Share Mode" button. When clicked, it reformats the UI into a perfect 9:16 aspect ratio box, hides all the web navigation elements, applies the sleekest styling, and literally prompts the user: *"Ready for the timeline. Take a screenshot to share."* It shifts the work to the user while maintaining the aesthetic.

If you encounter any of these blockers, execute the corresponding pivot without hesitation:

Blocker	**Pivot**
Deployment service requires browser auth	Try: Vercel CLI (vercel --yes) $\rightarrow$ Cloudflare (wrangler pages deploy) $\rightarrow$ Surge (surge dist/) $\rightarrow$ Netlify Drop (drag-and-drop) $\rightarrow$ GitHub Pages
Database service unavailable	Use localStorage for state persistence + JSON files for seed data. Note as limitation.
External API requires key you don't have	Mock the API response with realistic static data. Build the UI to consume real API shape so it's easy to swap later.

npm package fails to install	Find alternative package. If no alternative, implement the feature yourself.
Image hosting/CDN blocked	Use inline SVGs, DiceBear avatars (`api.dicebear.com`), UI Avatars (`ui-avatars.com`), or base64-encoded placeholders.
Font CDN blocked	Download fonts, bundle them locally in the project, and load via @font-face.
Build falls after changes	Read the error carefully. Fix the root cause. Do not revert all changes — isolate the problem.
Feature is taking 2x longer than allocated	Implement the 80% version. Ship it. Note the remaining 20% as a limitation.
You realize the architecture is wrong mid-build	If you're <30% done, refactor. If >30% done, work with what you have and note the architectural debt.

QUALITY STANDARDS

// These are the minimum quality bars. Customize per project.

Code quality

- Clean, readable code with meaningful variable/function names
- Components are modular and reusable
- No hardcoded values where variables should exist (colors, spacing, copy)
- TypeScript preferred (if using JS framework). At minimum, add JSDoc types.
- No `any` types in TypeScript (use proper interfaces)

- No commented-out dead code in the final build

Design quality

- Consistent spacing system (4px/8px base grid)
- Typography hierarchy is clear (no two text elements should look the same unless they serve the same purpose)
- Color palette is cohesive (3-5 colors max, used intentionally)
- Interactive elements have hover, active, focus, and disabled states
- Animations serve a purpose (feedback, orientation, delight) — not decoration
- Dark mode preferred unless the project specifies otherwise
- NEVER use default/generic AI aesthetics (Inter font, purple gradients, rounded rectangles with drop shadows)

UX quality

- Every screen has a clear primary action
- Empty states are designed (not blank white screens)
- Loading states exist for any async operation
- Error states are helpful ("Something went wrong" is not acceptable — say what happened and what to do)
- Forms auto-save when possible
- Navigation is obvious and consistent
- Mobile-first — the app must work on a phone

Performance

- Initial load under 3 seconds
- Lighthouse performance score > 80
- No render-blocking resources
- Images optimized (lazy-load below the fold)
- Fonts loaded with `display=swap`

REMINDER

You have 7 hours. That is a lot of time. Use it wisely:

- Don't skip research. Understand what you're building before you build it.
- Don't skip design. A beautiful product earns trust. An ugly product doesn't.
- Don't skip testing. Every untested feature is a bug waiting to happen.
- Don't skip polish. Favicons, meta tags, loading states, empty states — these are the difference between "prototype" and "product."

The only message I want from you is the final deliverable. Make it count. **Go.**